

repmat

Repeat copies of array

Syntax

```
B = repmat(A,n)
B = repmat(A,r1,...,rN)
B = repmat(A,r)
```

Description

`B = repmat(A,n)` returns an array containing `n` copies of `A` in the row and column dimensions. The size of `B` is `size(A)*n` when `A` is a matrix. [example](#)

`B = repmat(A,r1,...,rN)` specifies a list of scalars, `r1,...,rN`, that describes how copies of `A` are arranged in each dimension. When `A` has `N` dimensions, the size of `B` is `size(A).*[r1...rN]`. For example, `repmat([1 2; 3 4],2,3)` returns a 4-by-6 matrix. [example](#)

`B = repmat(A,r)` specifies the repetition scheme with row vector `r`. For example, `repmat(A,[2 3])` returns the same result as `repmat(A,2,3)`. [example](#)

Examples

[collapse all](#)

Initialize Matrix with Same Element Value

Create a 3-by-2 matrix whose elements contain the value 10.

[Open in MATLAB Online](#)[Copy Command](#)

```
A = repmat(10,3,2)
```

[Get](#) ▼

```
A = 3x2
```

```
10    10
10    10
10    10
```

Square Block Format

Repeat copies of a matrix into a 2-by-2 block arrangement.

[Open in MATLAB Online](#)

 Copy Command

```
A = diag([100 200 300])
```

 Get ▾

```
A = 3×3
```

100	0	0
0	200	0
0	0	300

```
B = repmat(A,2)
```

 Get ▾

```
B = 6×6
```

100	0	0	100	0	0
0	200	0	0	200	0
0	0	300	0	0	300
100	0	0	100	0	0
0	200	0	0	200	0
0	0	300	0	0	300

Rectangular Block Format

Repeat copies of a matrix into a 2-by-3 block arrangement.

[Open in MATLAB Online](#) Copy Command

```
A = diag([100 200 300])
```

 Get ▾

```
A = 3×3
```

100	0	0
0	200	0
0	0	300

```
B = repmat(A,2,3)
```

 Get ▾

```
B = 6×9
```

100	0	0	100	0	0	100	0	0
0	200	0	0	200	0	0	200	0
0	0	300	0	0	300	0	0	300
100	0	0	100	0	0	100	0	0
0	200	0	0	200	0	0	200	0
0	0	300	0	0	300	0	0	300

3-D Block Array

Repeat copies of a matrix into a 2-by-3-by-2 block arrangement.

[Open in MATLAB Online](#)
[Copy Command](#)

```
A = [1 2; 3 4]
```

[Get](#) ▼

A = 2×2

```
1    2
3    4
```

```
B = repmat(A,[2 3 2])
```

[Get](#) ▼

B =

B(:, :, 1) =

```
1    2    1    2    1    2
3    4    3    4    3    4
1    2    1    2    1    2
3    4    3    4    3    4
```

B(:, :, 2) =

```
1    2    1    2    1    2
3    4    3    4    3    4
1    2    1    2    1    2
3    4    3    4    3    4
```

Vertical Stack of Row Vectors

Vertically stack a row vector four times.

[Open in MATLAB Online](#)
[Copy Command](#)

```
A = 1:4;
```

```
B = repmat(A,4,1)
```

[Get](#) ▼

B = 4×4

```
1    2    3    4
1    2    3    4
```

1	2	3	4
1	2	3	4

Horizontal Stack of Column Vectors

Horizontally stack a column vector four times.

[Open in MATLAB Online](#)
[Copy Command](#)

```
A = (1:3)';
B = repmat(A,1,4)
```

[Get](#)

B = 3x4

1	1	1	1
2	2	2	2
3	3	3	3

Tabular Block Format

Create a table with variables Age and Height.

[Open in MATLAB Online](#)
[Copy Command](#)

```
A = table([39; 26],[70; 63], 'VariableNames',{'Age' 'Height'})
```

[Get](#)

A=2x2 *table*

Age	Height
39	70
26	63

Repeat copies of the table into a 2-by-3 block format.

```
B = repmat(A,2,3)
```

[Get](#)

B=4x6 *table*

Age	Height	Age_1	Height_1	Age_2	Height_2
39	70	39	70	39	70
26	63	26	63	26	63

39	70	39	70	39	70
26	63	26	63	26	63

repmat repeats the entries of the table and appends a number to the new variable names.

Combine Vector Elements

Create two column vectors.

Open in MATLAB
Online

 Copy Command

```
A = [1; 3; 5];
B = [2; 4];
```

 Get ▾

Generate all element combinations of the two vectors by using `repelem` and `repmat`. Each row of the output `T` is a combination with the first element coming from the first vector and the second element coming from the second vector. This command is equivalent to finding the Cartesian product of two vectors.

```
T = [repelem(A,numel(B)) repmat(B,numel(A),1)]
```

 Get ▾

`T = 6×2`

1	2
1	4
3	2
3	4
5	2
5	4

Starting in R2023a, you can also use the [combinations](#) function to generate all element combinations of two vectors.

```
T = combinations(A,B)
```

 Get ▾

`T=6×2 table`

A	B
—	—
1	2
1	4
3	2
3	4
5	2
5	4

Input Arguments

A — Input array

scalar | vector | matrix | multidimensional array

Input array, specified as a scalar, vector, matrix, or multidimensional array.

Data Types: single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64 | logical | char | string | struct | table | datetime | duration | calendarDuration | categorical | cell

Complex Number Support: Yes

n — Number of times to repeat input array in row and column dimensions

integer value

Number of times to repeat the input array in the row and column dimensions, specified as an integer value. If n is 0 or negative, the result is an empty array.

Data Types: single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

r1, ..., rN — Repetition factors for each dimension (as separate arguments)

integer values

Repetition factors for each dimension, specified as separate arguments of integer values. If any repetition factor is 0 or negative, the result is an empty array.

Data Types: single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

r — Vector of repetition factors for each dimension (as a row vector)

integer values

Vector of repetition factors for each dimension, specified as a row vector of integer values. If any value in r is 0 or negative, the result is an empty array.

Data Types: single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

Tips

- To build block arrays by forming the tensor product of the input with an array of ones, use [kron](#). For example, to stack the row vector $A = 1:3$ four times vertically, you can use $B = \text{kron}(A, \text{ones}(4,1))$.
- To create block arrays and perform a binary operation in a single pass, use [bsxfun](#). In some cases, `bsxfun` provides a simpler and more memory efficient solution. For example, to add the vectors $A = 1:5$ and $B = (1:10)'$ to produce a 10-by-5 array, use `bsxfun(@plus,A,B)` instead of `repmat(A,10,1) + repmat(B,1,5)`.
- When A is a scalar of a certain type, you can use other functions to get the same result as `repmat`.

repmat Syntax	Equivalent Alternative
<code>repmat(NaN,m,n)</code>	<code>NaN(m,n)</code>
<code>repmat(single(inf),m,n)</code>	<code>inf(m,n,'single')</code>

repmat Syntax	Equivalent Alternative
<code>repmat(int8(0),m,n)</code>	<code>zeros(m,n,'int8')</code>
<code>repmat(uint32(1),m,n)</code>	<code>ones(m,n,'uint32')</code>
<code>repmat(eps,m,n)</code>	<code>eps(ones(m,n))</code>

Extended Capabilities

expand all

Tall Arrays
Calculate with arrays that have more rows than fit in memory.

C/C++ Code Generation
Generate C and C++ code using MATLAB® Coder™.

GPU Code Generation
Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

HDL Code Generation
Generate VHDL, Verilog and SystemVerilog code for FPGA and ASIC designs using HDL Coder™.

Thread-Based Environment
Run code in the background using MATLAB® backgroundPool or accelerate code with Parallel Computing Toolbox™ ThreadPool.

GPU Arrays
Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

Distributed Arrays
Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

Version History

expand all

Introduced before R2006a

R2019b: Some repetition arguments produce error⚠️

See Also

[bsxfun](#) | [kron](#) | [repelem](#) | [reshape](#) | [resize](#) | [paddata](#) | [meshgrid](#) | [ndgrid](#)

YOU MIGHT ALSO BE INTERESTED IN
[Converting Deep Learning Models Between PyTorch, TensorFlow, and MATLAB](#)

Import and export AI models between PyTorch®, TensorFlow™, and MATLAB®.

[Learn more](#)